Problem Solving Using Python (CSE1012)

Dr. Abdul Kayom, Assistant Professor, SENSE

Agenda / Contents

Recursion

- A recursive function is a function that calls itself

- The call is made until it doesn’t require a call to itself to solve the particular problem

Recursion

- Every recursive function has two major cases:
  - Base case: in which the problem is simple enough to solve without making any further calls to the same function
  - Recursive case:
    - The problem is divided into smaller sub-parts
    - The function calls itself
    - The result is obtained by combining all the sub-parts

Recursion Example

- To understand recursion, take the example of calculating factorial
- n!=n*(n-1)*(n-2)*…*1
- Or, n!=n*(n-1)!

- factorial(n)=n * factorial(n-1)

Recursion Example

- Compute factorial of a number by using recursion

Recursion Example

```
#Factorial Calculation using Recursion
def fact(n):
    if(n==1 or n==0):
        return 1
    else:
        return n*fact(n-1)
n=int(input("Enter the Number n: "))
fact_val=fact(n)
print("The factorial of",n,"is: ",fact_val)
```

Steps for Recursion

step1: specify the base case which will stop the function from making a call to itself

step2: check if the current value being processed matches with the value of the base case. If yes, process and return the value

step3: divide the problem into simpler sub-problem

step4: call the function on the sub-problem

step5: combine the results of the sub-problem

step6: return the final result

Recursion

- The base case of a recursive function acts as the termination condition
- So, in the absence of an explicitly defined base case, a recursive function would call itself indefinitely

Greatest Common Divisor(GCD)

- GCD of two numbers (integer) is the largest integer that divides both the numbers

- We can find GCD of two numbers recursively by using the Euclid’s algorithm

Greatest Common Divisor(GCD)

- Euclid’s algorithm states that:
- GCD(a,b)=
  - b, if b divides a
  - GCD(b, a%b), otherwise

- Assuming a>b, (interchange a, b if b>a)

Greatest Common Divisor(GCD)

#Write a function to Calculate GCD

Greatest Common Divisor(GCD)

```
#GCD Calculation
def GCD(x,y):
    if y>x:
        (x,y)=(y,x)
    rem=x%y
    if(rem==0):
        return y
    else:
        return GCD(y,rem)
m=int(input("Enter the first number: "))
n=int(input("Enter the second number: "))
GCD_val=GCD(m,n)
print("The GCD of",m,"and",n,"is: ",GCD_val)
```

Finding Exponent(x^y)

- We can find a solution to find exponent of a number using recursion
- To find x^y, the base case would be when y=0

EXP(x,y)=

1, if y==0

x* EXP(x,y-1), otherwise

Finding Exponent(x^y)

```
#Calculate Exp(x,y)
def EXP(x,y):
    if(y==0):
        return 1
    else:
        return (x*EXP(x,y-1))
m=int(input("Enter the first number: "))
n=int(input("Enter the second number: "))
EXP_val=EXP(m,n)
print("The Result is",EXP_val)
```

Fibonacci Series

- The fibonacci series is given as,

0 1 1 2 3 5 8 13 21 34 55…

- i.e. nth term=(n-1)th term + (n-2)th term

Fibonacci Series

- FIB(n)=

  1, if n<=2

  FIB(n-1) + FIB(n-2), otherwise

Fibonacci Series

```
#Fibonacci Series using Recursion
def FIB(n):
    if(n==0):
        return 0
    if(n==1):
        return 1
    else:
        return (FIB(n-1)+FIB(n-2))
n=int(input("Enter the number of terms: "))
for i in range(n):
     print("The Fibonacci Series is: ",FIB(i))
```

Recursion Vs Iteration

- Recursion is more of a top-down approach to problem solving
- The original problem is divided into smaller sub problems
- Iteration follows a bottom-up approach
- It begins with what is known and then constructing the solution step-by-step

Benefits of using Recursion

- Recursive solution often tend to be shorter and simpler than non-recursive ones
- Code is clearer and easier to use
- It uses the original formula to solve a problem
- It uses divide and conquer technique to solve a problem

- Recursive functions make the code look clean and elegant

Limitations of using Recursion

- Aborting a recursive process in midstream is slow and sometimes nasty
- Takes more time and space for execution
- Difficult to find bugs, particularly when using global variables

Thank You

Dr. Abdul Kayom


Asst. Prof, (SENSE)


kayomabdul@vitap.ac.in


8474860025